

Comparative Analysis of Algorithms in Constrained Portfolio Optimization

Larissa Chee Shuen Yui
School of Mathematical Science
University of Nottingham Malaysia
szylc2@nottingham.edu.my

Kai Zheng Kan
School of Mathematical Science
University of Nottingham Malaysia
szykk1@nottingham.edu.my

Jia Feng Sih
School of Mathematical Science
University of Nottingham Malaysia
szyjs1@nottingham.edu.my

Abstract - Portfolio optimization is a widely studied optimization problem, particularly in institutional investing, quantitative finance, and wealth management. Although historically solvable using quadratic programming methods, various types of algorithms have been explored to address the increased complexity of the optimization problem as researchers and practitioners seek to make it more applicable to real-world scenarios. This paper presents a historical overview of algorithms used to solve portfolio optimization problems and implements and compares 3 algorithms: Simulated Annealing (SA), Genetic Algorithms (GA), Bayesian Optimization (BO) with the objective of maximising the Sharpe Ratio. The Mean Variance Model is used as a benchmark. Results show that GA performed with the best Sharpe Ratio, though it had the longest runtime.

Keywords - Portfolio Optimization, Mean-Variance Optimization (MVO), Simulated Annealing (SA), Genetic Algorithms (GA), Bayesian Optimization (BO), Metaheuristics, Constrained Optimization, Sharpe Ratio, VaR (Value-at-Risk), CVaR (Conditional Value-at-Risk), Risk Management, Computational Efficiency, Convergence, Asset Allocation, Statistical Analysis

I. INTRODUCTION

Portfolio optimization involves selecting the optimal combination of financial assets to achieve a specific investment objective—usually maximizing expected returns for a given level of risk, or minimizing risk for a given return [1]. This framework was first formalized by Harry Markowitz in 1952 with the introduction of Modern Portfolio Theory (MPT), which highlights the trade-off between risk and return [2]. While finding a portfolio that minimizes risk or maximizes returns can be solved using exact methods (such as Mean-Variance Optimization (MVO) [2]) in polynomial time, this approach is often considered an oversimplification of real-world investing challenges. Over time, additional real-world constraints, such as Value-at-Risk (VaR) constraints, have been incorporated into the basic MPT model. However, these constraints increase the complexity of portfolio optimization, making it an NP-Hard problem [3]. Consequently, solving such problems requires more sophisticated techniques, such as metaheuristics [4] and approximation algorithms [5], which are employed in this study.

This paper begins by discussing the different investment objectives and algorithms historically used for portfolio optimization. It then details the implementation of three of these algorithms for constrained portfolio optimization problems on 10 S&P500 stocks and evaluates their performances, using the MVO as the benchmark.

II. LITERATURE REVIEW

The MVO[2], otherwise known as the Markowitz Model, is the first widely recognized and formalized

approach to portfolio optimization typically solved using quadratic programming. Despite its long legacy, the MVO is still a subject study in recent years, analysing its advantages and limitations [6],[7]. At the same time, many studies have also been done applying different methods to overcome some of its limitations. In 2024, Dogan et al. carried out portfolio optimization using the local search metaheuristic Simulated Annealing (SA), since it is able to handle higher dimensional problems compared to the MVO [8]. Evolutionary algorithms like Genetic Algorithms (GA) have also been applied to portfolio optimization in an endeavor to avoid making the plausibly inaccurate assumption of normally distributed assets made in the MVO [9]. More recently, Bayesian Optimization (BO) has also been attempted to capture the uncertainty associated with the problem [10]. These are but among the few methods that were implemented in an attempt to overcome the limitations of the traditional MVO model, creating the potential for these methods to be compared against each other. For example, in 2015, a study was made by Ntumba comparing the performance of SA and GA for portfolio optimization [11]. Although investment objectives and hence, their complexity are often dependent on the investor's preference, one common objective would be to maximise the Sharpe Ratio, a metric developed by Sharpe in 1966 to reflect the expected gain obtained per unit risk taken [12]. For example, in 2025, Keon published a study involving Sharpe Ratio-Based Optimization [13].

III. METHODOLOGY

In this study, portfolio optimization is performed using Simulated Annealing (SA), Genetic Algorithms (GA), and Bayesian Optimization (BO) on a fixed set of 10 S&P 500 stocks. The details regarding the a) choice of algorithms, b) implementation methods, c) evaluation methods will be provided in the methodology.

A. Choice of Algorithms

Portfolio optimization problems often become non-convex when real-world constraints or objectives (e.g. transaction costs, cardinality limits, Value-at-Risk (VaR), Sharpe ratio) are introduced [14]. This non-convexity is a critical consideration when selecting an appropriate optimization algorithm. In such settings, it is also essential that the algorithm can avoid suboptimal solutions by escaping local optima. Simulated Annealing (SA), Genetic Algorithms (GA), and Bayesian Optimization (BO) are well-suited for these challenges due to their stochastic search mechanisms, which enable them to explore non-convex, discontinuous, or rugged solution spaces effectively and search broadly across the solution landscape, escaping local optima as will be demonstrated in Section 4. The table below summarizes their classification and suitability for non-convex optimization.

TABLE I : ALGORITHM CLASSIFICATION

Algorithm	Solution Strategy	Search Strategy	Method	Algorithm Type
SA	Improvement	Global	Stochastic	Meta-heuristic
GA	Improvement	Global	Stochastic	Meta-heuristic
BO	Model-based	Global	Stochastic	Approximation Algorithm

B. Implementation methods

1) General Operators

Generating Random Solutions

All three algorithms — particularly Bayesian Optimization — require valid random portfolio solutions. A unified method is used to generate such solutions despite the complexity introduced by constraints on the solution space. First, a random point is sampled from a Dirichlet distribution to ensure the weights sum to one. This point is then projected into the feasible region by solving an optimization problem that minimizes the distance to the original sample while satisfying the asset bounds (using the `cvxpy` library). Finally, starting from this projected point, a Hit-and-Run sampler is used to explore the feasible region until a solution is found that also meets the CVaR and VaR constraints.

Objective/Fitness function

The Sharpe Ratio is defined as:

$$\text{Sharpe Ratio} = \frac{r^T w - r_f}{\sqrt{w^T \Sigma w}}$$

(1)

where r is the vector of expected asset returns, w is the vector of portfolio weights, Σ is the variance-covariance matrix of returns, and r_f is the risk-free rate (note that $r^T w$ gives the expected portfolio return and $\sqrt{w^T \Sigma w}$ is the standard deviation of the portfolio returns). This metric evaluates the trade-off between risk and return, indicating how much excess return is earned per unit of risk.

In this study, portfolio optimization aims to maximize the Sharpe Ratio, which serves as the fitness function for all three algorithms—SA, GA, and BO. By doing so, the optimization inherently favors portfolios that deliver higher returns while controlling risk, in line with the principles of Modern Portfolio Theory [2].

2) Data Extraction and Preparation

Historical stock data from Yahoo Finance (2010-2025) was retrieved for 10 S&P 500 stocks: NVIDIA, Tesla, Broadcom, Texas Pacific Land Trust, Netflix, Axon Enterprise, Fair Isaac, United Rentals, Transdigm, and Domino's Pizza. The 10-year U.S. Treasury yield was used as a proxy for the risk-free rate. Each stock's return and the variance-covariance matrix were calculated, which are used to compute the Sharpe ratio.

3) Constraints

The constraints imposed on the portfolio optimization and the conditions used to impose them are:

1. Budget constraint: sum of weights = 1
2. No short selling: weights ≥ 0
3. Upper & Lower Bounds on Asset Allocation:

4. Risk Constraints: VaR and CVaR values are limited to ensure the portfolio does not exceed a certain level of risk

In this study, the upper and lower bounds are set to 0.05 and 0.5 respectively, so for every weight, $0.05 \leq \text{weight} \leq 0.5$. In other words, the proportion of the budget allocated to each asset should be between 5% and 50%. This naturally ensures that no short selling occurs, since every weight will definitely be greater than zero.

The limits for VaR and CVaR are usually set by the investor. Here, these limits are set using a function that attempts to simulate some of the procedures taken by an investor when determining the VaR and CVaR limits. More specifically, 5000 valid weights are generated and the limits are set based on the top 10% lowest risk portfolio. All the constraints above will be considered hard constraints. Additionally, an all-feasible approach will be taken when comparing the algorithms. This approach allows for a more comprehensive comparison between the results from the different models.

4) Algorithm Specific Implementation Methods

Mean-Variance Optimization with Auto-Tuned VaR/CVaR Limits

As mentioned above, the mean-variance optimization (Markowitz Model) is used as a benchmark for comparison in this study. Therefore, the implementation will only be briefly described.

The process begins by optimizing the portfolio for 50 evenly spaced target return values. Specifically, the goal is to minimize portfolio variance using the `cvxpy` library, subject to the following constraints:

- The sum of portfolio weights equals 1
- Weights are within specified lower and upper bounds
- The expected portfolio return meets or exceeds the target return

The resulting 50 optimized portfolios form the efficient frontier. From this set, portfolios that satisfy the VaR and CVaR thresholds are identified. Among these, the portfolio with the highest Sharpe ratio is selected as the optimal solution.

Simulated Annealing

Require: Initial temperature T_0 , minimum temperature T_{min} , maximum iterations N , decay parameters α, β , schedule type

Ensure: Best portfolio weights and corresponding score

```

1: Set random seed (if provided)
2: Initialize a valid solution and evaluate its score
3: Set best solution as the initial one
4: Initialize temperature  $T \leftarrow T_0$ 
5: for iteration = 1 to  $N$  do
6:   Generate neighbor solution by altering current one
7:   Evaluate neighbor solution's score
8:   Compute fitness difference  $\Delta$ 
9:   Accept neighbor if  $\Delta > 0$  or with probability  $\exp(\Delta/T)$ 
10:  Update best solution if current one improves it
11:  Update temperature according to schedule:
    • Linear:  $T \leftarrow T - T_0/N$ 
    • Geometric:  $T \leftarrow \alpha T$ 
    • Lundy-Mees:  $T \leftarrow T/(1 + \beta T)$ 
    • ConstExp2:  $T \leftarrow \text{constant then exponential}$ 
12:  if  $T < T_{min}$  then
13:    Break
14:  end if
15: end for
16: return Best solution and its score

```

Fig .1. Pseudocode for Simulated Annealing

Simulated Annealing is a local search metaheuristic first proposed by Kirkpatrick et al. (1983)[15] that mimics the annealing process. The implementation of

this concept used in this study is detailed in the pseudocode from Fig. 1 above. The adaptation of the SA algorithm for portfolio optimization can be broken up into a few parts:

1. Initialization

The initialization of both the initial solution and the temperature are critical aspects of Simulated Annealing (SA). Following the approach of Xiao (2011)[16], a valid initial solution is randomly generated using the method described in Section D. If not predefined, the initial temperature is set to a high default value of 1000. However, through manual tuning, we find that an initial temperature of 1000 tends to produce better results in practice.

2. Generating Neighbour Solution

Neighboring solutions are generated using a method adapted from Chang et al. (2000)[17]. It works by:

- Randomly selecting two asset weights, i and j
- Calculating how much i can increase and j can decrease without violating bounds
- Using the smaller of these as the maximum transferable amount
- Transferring a random portion of this amount from j to i

This process repeats until a neighbor is found that satisfies both the VaR and CVaR constraints.

3. Acceptance criterion

The Metropolis criterion is employed as the acceptance mechanism (line 8, Fig. 1). As described by van Laarhoven and Aarts (1987)[18], the criterion is derived from the Boltzmann distribution and the Monte Carlo method, which originate in statistical thermodynamics. Specifically, the Boltzmann factor governs the probability of transitioning between states in a thermodynamic system, and is adapted here to guide the acceptance of new solutions in the optimization process. According to this criterion, a newly generated neighboring solution is accepted if it either improves the objective function, or—if it is worse—is accepted with a probability given by:

$$P_{\text{accept}} = \exp\left(-\frac{E_{\text{new}} - E_{\text{current}}}{T}\right) \quad (2)$$

where E_{new} and E_{current} are the objective values of the new and current solutions, respectively, and T is temperature parameter. As noted by Amine (2019)[19], the allowance for accepting non-improving solutions facilitates diversification in the search process and enables escape from local optima.

4. Cooling Schedule

Four different cooling schedules were tested: linear, geometric (with rate of 0.95), Lundy and Mees from (Lundy & Mees, 1986)[20] with $\beta = 0.95$ and ConstExp2 , a schedule proposed by Velleda Gonzales et al. (2015)[21] which demonstrated strong performance. Lundy and Mees was selected for its consistency, and highest average result which was achieved in a reasonable run time.

5. Stopping Criterion

The algorithm is stopped either when the maximum iteration has reached or when the temperature has converged to a sufficiently small value $-1e-6$ - set using trial and error.

Genetic Algorithms

Require: Historical returns matrix, mean returns μ , covariance matrix Σ , risk-free rate r_f , population size, number of generations, mutation rate, elite size, patience

Ensure: Optimal weights and corresponding Sharpe ratio

- 1: **Set random seed** (if provided)
- 2: Initialize population of candidate portfolios
- 3: Set best Sharpe ratio to $-\infty$
- 4: **For** each generation $g = 1$ to max generations **do**
- 5: Evaluate fitness of each individual using Sharpe ratio
- 6: Filter out individuals violating VaR and CVaR constraints
- 7: Update best solution if improved
- 8: **If** no improvement for given patience, **break**
- 9: Select parents using tournament selection
- 10: Preserve elite individuals
- 11: Generate offspring via crossover
- 12: Apply mutation to offspring
- 13: Form new population from elites and offspring
- 14: **End For**
- 15: **If** no valid solution found, return equal weights
- 16: **return** Best portfolio weights and corresponding Sharpe ratio

Fig .2. Pseudocode for Genetic Algorithm

The GA, an algorithm by natural selection and evolution, was first initiated by Holland (1992)[22]. The pseudocode in Fig. 2 outlines the application of GA in this study.

1. Initialization

The GA starts by randomly initializing portfolio weights using the method described in Section D. This randomness encourages exploration and helps prevent early convergence, following common practice in evolutionary algorithms. [23]

2. Tournament Selection

Tournament selection is used for parent selection, where a small group of individuals (size=3) is randomly chosen, and the one with the best fitness (highest Sharpe Ratio) is selected. This method balances exploration and exploitation, avoids fitness scaling problems, and helps maintain diversity, reducing the risk of getting stuck in poor solutions. [24]

3. Crossover

The crossover uses the BLX- α (Blend Crossover) method with $\alpha = 0.5$, creating offspring by sampling slightly beyond the parent values. This helps explore the search space and maintain diversity, making it well-suited for real-valued problems like portfolio optimization. [25]

4. Mutation

Mutation adds Gaussian noise to 30% of the portfolio weights. The mutated solution is then projected into the feasible region (satisfying bounds and summing to one) using a constrained optimization problem with the cvxpy library. This fixed rate ensures consistent exploration during the search, unlike adaptive schemes. [26]

5. Elitism

An elitism strategy keeps the top 10% of individuals unchanged in the next generation. This preserves the best solutions, helping the algorithm converge faster and stay more stable over time. [27]

6. Hybrid Optimization

After the Genetic Algorithm finishes, the best solution is refined using the Trust-Constr optimizer, a gradient-based method. This hybrid approach combines GA's global search with accurate local optimization, improving solution quality and reliability—an approach well-supported for constrained problems. [28]

Bayesian Optimization

Require: Mean returns μ , covariance matrix Σ , risk-free rate r_f , number of iterations n_{iter} , proposal std σ , Dirichlet prior coefficient α

Ensure: Accepted weights, Sharpe statistics, and constraint thresholds

- 1: **Input validation** ▷ Check dimension and parameter validity
- 2: Generate initial feasible solution using Dirichlet distribution
- 3: **while** initial solution violates VaR/CVaR constraints **do**
- 4: Resample using Hit-and-Run
- 5: **end while**
- 6: Initialize current solution and compute Sharpe ratio
- 7: Initialize list of accepted weights and Sharpe ratios
- 8: **for** each iteration $i = 1$ to n_{iter} **do**
- 9: **while** proposal violates VaR/CVaR constraints **do**
- 10: Propose new weights via Hit-and-Run sampling
- 11: **end while**
- 12: Compute Sharpe ratio of proposed weights
- 13: Compute prior probability using Dirichlet prior
- 14: Compute log acceptance ratio
- 15: Accept or reject based on Metropolis-Hastings rule
- 16: **end for**
- 17: **if** too few valid samples accepted **then**
- 18: Raise warning or error
- 19: **end if**
- 20: Apply burn-in to accepted samples
- 21: Compute mean and standard deviation of post-burn-in Sharpe ratios
- 22: Identify best sample and top 0.1% performers
- 23: **return** Accepted weights, Sharpe stats, best and top-performing portfolios, VaR and CVaR limits

Fig .3. Pseudocode for Bayesian Portfolio Optimization

Bayesian Optimization is a sample-efficient method suited for problems where the objective is costly to evaluate or lacks a clear formula. Instead of relying on gradients, it builds a probabilistic model (usually a Gaussian Process) to estimate the objective and uses acquisition functions to balance exploration and exploitation. This makes it ideal for optimizing black-box functions with constraints or uncertainty.

1. Initial and Proposal Solution Generation

Random sampling plays a key role in Bayesian Optimization (BO) by guiding exploration early on, helping avoid premature convergence and improving overall performance. In this study, both the initial and proposal solutions are randomly generated using the method described in Section D. Salgia et al. (2023)[29] emphasize the value of random sampling in BO, showing that it improves both accuracy and efficiency.

2. Prior Distribution

The prior captures our beliefs about feasible portfolio weights. Here, a Dirichlet distribution is used because it naturally generates weight vectors that are positive and sum to one—key requirements in portfolio construction¹. [30] Different settings of the concentration parameter α allow shaping the prior: [31]

- $\alpha = (1, 1, \dots, 1)$: uniform prior (no preference)
- $\alpha = (5, 5, \dots, 5)$: diversified portfolios
- $\alpha = (0.5, 0.5, \dots, 0.5)$: sparse portfolios
- $\alpha = (10, 1, \dots, 1)$: preference for the first asset

In this study, a symmetric prior with equal α values is used, resulting in a uniform distribution.

3. Acceptance-Rejection Step

The acceptance-rejection mechanism in the Metropolis-Hastings algorithm decides whether a newly proposed portfolio should be accepted. The acceptance ratio is calculated as:

$$\text{Acceptance Ratio} = \frac{\text{Sharpe}(\text{proposed}) \cdot \text{Prior}(\text{proposed})}{\text{Sharpe}(\text{current}) \cdot \text{Prior}(\text{current})} \quad (3)$$

To avoid numerical instability from very small values, we take the logarithm of the acceptance ratio :

$$\log(\text{Acceptance Ratio}) = \log(S_{\text{new}}) + \log(P_{\text{new}})$$

$$- \log(S_{\text{current}}) - \log(P_{\text{current}})$$

(4)

Next, we generate a random value u from a uniform distribution between 0 and 1. The proposed solution is accepted if:

$$\log(u) < \log(\text{Acceptance Ratio})$$

If not, the current weights are retained. Importantly, the proposal distribution term $q(x|x')q(x'|x)$ is excluded, as we assume a symmetric proposal distribution (e.g., Dirichlet with equal α values). This symmetry ensures the term cancels out, simplifying the calculation, a common practice in Bayesian optimization. [32]

4. Burn-in Step

To ensure the Markov Chain reaches its stationary distribution, the first 20% of iterations are discarded as burn-in. This removes the influence of the initial state, ensuring the remaining samples more accurately represent the true posterior distribution.

5. Result Analysis

After collecting the posterior samples, the mean and standard deviation of the Sharpe Ratios are calculated to understand the expected performance and variability. The top 0.01% of portfolios with the highest Sharpe Ratios (around 8 portfolios) are then selected as the best solutions. These elite samples represent optimal portfolios that balance return and risk under the Bayesian framework.

C. Choice of evaluation methods

Finally, the result of the test will be evaluated and discussed based on their Sharpe ratio, cumulative returns, runtimes and fitness traces. A Wilcoxon signed rank test will also be carried out to test the significance of some results.

IV. RESULTS AND DISCUSSION

The simulations were conducted on a 13th Gen Intel® Core™ i7-13700H CPU (2.40 GHz) with 16 GB RAM. All algorithms were implemented in Python 3.12 and executed within a controlled environment. The overall performance of four portfolio optimization methods—MVO, SA with Lundy and Mees (L&M) cooling Schedule, GA and BO—was evaluated across 5 independent trials with 10000 iterations. Evaluation metrics include Sharpe Ratio, cumulative returns, volatility, and runtime. The statistical results are summarized in TABLE II.

TABLE II. PERFORMANCE COMPARISON OF PORTFOLIO OPTIMIZATION ALGORITHM

	Algorithm			
	MVO	SA (L&M)	GA	BO
Sharpe Ratio	1.7236±0.0000	1.7241±0.0002	1.8422±0.0111	1.6235±0.0106
Cumulative Return (%)	41.35±0.00	15210.62±125.62	13012.03±434.41	16954.55±1199.49
Volatility (%)	22.44±0.00	22.52 ± 0.04	20.14 ± 0.29	24.72 ± 0.53
Runtime (s)	—	0.98 ± 0.04	61.04±26.41	0.93±0.02

From the results in Table II we find that GA performs with the highest Sharpe Ratio of around 1.8422,

while MVO and SA performed similarly, with sharpe ratios of 1.7236 and 1.7241 respectively, BO had by far the poorest sharpe ratio of only 1.6235. This pattern is reflected in the levels of volatility as well.

GA's higher Sharpe Ratio is explainable, since unlike BO and SA, GA uses a population-based search strategy, which allows it to explore multiple regions of the search space in parallel, which increases the chances of avoiding local optima in complex spaces as defined in this study. We find the GA produced a more diverse spread of solutions.

SA's exploration scheme - where exploration is gradually replaced with exploitation as randomness is reduced over time - is likely the reason for its convergence towards a comparatively lower Sharpe ratio on average. Nonetheless, this scheme can also be credited for its predictable performance (lowest standard deviation of 0.0002). Whether this is a worthy trade off between stability and global optimality largely depends on the preference of an investor.

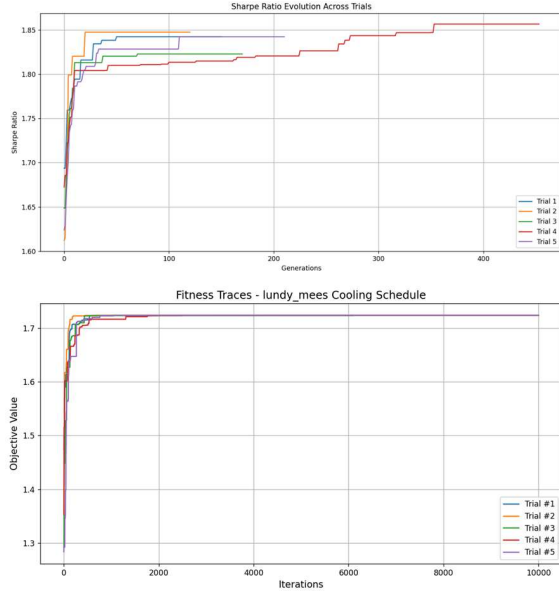


Fig .4. Convergence of Sharpe Ratio for SA and GA

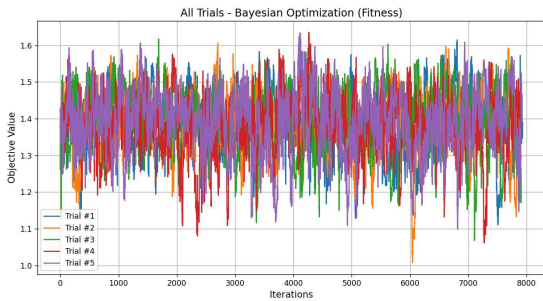


Fig .5. Sharpe Ratio Evolution for BO across Trials

BO's poor results may be attributed to several factors. One possible explanation is that the algorithm has not yet fully converged. Unlike the fitness traces for GA and SA shown in Fig. 4, the fitness trace for BO (Fig. 5) suggests that convergence has not been achieved. However, increasing the number of iterations to 100,000 (where BO converged) yielded only a marginal improvement, with the average Sharpe Ratio remaining at 1.6235 ± 0.0106 . This

suggests that BO may have converged to a local optimum, potentially due to suboptimal choices of acquisition functions or surrogate models. Further investigation—beyond the scope of this study—is necessary to fully explain BO's underperformance.

Nonetheless, it is important to note that BO's underperformance in terms of Sharpe ratio compared to MVO does not indicate that it is conceptually still less suitable for non-convex portfolio optimization problems than MVO. This is because while MVO is able to produce a feasible result of reasonable quality, the assumptions it makes don't hold in non-convex settings, hence the "better" results produced could be fragile or misleading.

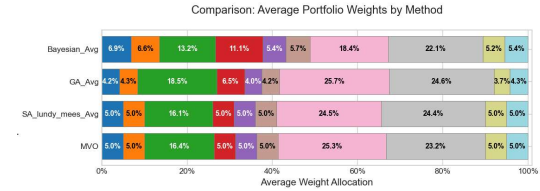


Fig .6. Comparison of Average Portfolio Weights by Algorithms

These average Sharpe Ratio results, when analyzed alongside their corresponding average portfolio weights, offer additional insight into the role of specific stocks in the optimization process. From Fig. 6, it is evident that the Genetic Algorithm (GA), which achieved the highest average Sharpe Ratio, allocated a larger portion of the portfolio to DPZ, TDG, and TPL. In contrast, Bayesian Optimization (BO), which recorded the lowest Sharpe Ratio, assigned the smallest weights to these same stocks. This suggests that greater investment in DPZ, TDG, and TPL may have been a key driver of improved risk-adjusted performance within the constraints of this study.

However, by analysing the runtimes of these algorithms, the trade-off between computational efficiency and quality of the solution is clear, where algorithms with higher Sharpe ratios also come with longer run times. Specifically, GA, with the highest Sharpe Ratio, has the longest run time of 61.04 ± 26.41 s. Conversely, BO had the lowest Sharpe Ratio, but only a run time of 0.93 ± 0.02 s.

A. Statistical Analysis

A statistical analysis using the non-parametric Wilcoxon signed-rank test was conducted on Sharpe Ratios and cumulative returns due to potential non-normality. This test assessed the significance of observed performance differences. The detailed pairwise comparison results are presented in TABLE III.

TABLE III. WILCOXON SIGNED-RANK TEST PAIRWISE COMPARISON RESULTS (N = 12 VALID PAIRS FOR ALL COMPARISONS AND $\alpha = 0.05$)

Algo. 1	Algo. 2	Wilcoxon Stat (W)	P-value	Significance
BO	GA	0	0.0005	Reject H0
BO	SA (LM)	0	0.0005	Reject H0
BO	MVO	0	0.0005	Reject H0
GA	SA (LM)	0	0.0005	Reject H0
GA	MVO	0	0.0005	Reject H0

SA (LM)	MVO	0	0.0005	Reject H0
---------	-----	---	--------	-----------

Results indicated statistically significant differences ($p < 0.05$) between MVO and all metaheuristic algorithms (SA, GA, BO) for both Sharpe Ratio and cumulative return. Furthermore, GA significantly outperformed SA and BO in Sharpe Ratio.

CONCLUSION

This study compared Simulated Annealing (SA), Genetic Algorithm (GA), and Bayesian Optimization (BO) for constrained portfolio optimization with MVO as the baseline. Results showed distinct performance trade-offs between speed and optimality: GA achieved the highest Sharpe Ratio but with the longest runtime, BO delivered the lowest Sharpe Ratio and shortest runtime, SA offered a consistent balance of competitive Sharpe Ratio and speed. While BO's Sharpe Ratio was worse than MVO, this does not indicate that BO is less suitable for complex portfolio optimization problems compared to MVO, which is fundamentally incompatible for non-convex spaces. Statistical analysis confirmed these performance distinctions were significant. In conclusion, the choice of algorithm thus depends on what an investor considers a 'worthy' trade-off. Future work could explore different datasets or algorithm parameter optimizations.

REFERENCES

- [1] A. Abdur, R. Khan, and L. K. Tanwani, "Portfolio optimization: Theory, methods, and applications," *ResearchGate*, Jun. 15, 2024. [Online]. Available: https://www.researchgate.net/publication/381458899_Portfolio_Optimization_Theory_Methods_and_Applications
- [2] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, Mar. 1952, doi: <https://doi.org/10.2307/2975974>.
- [3] Z. Tang, A. L. Dou, and A. K. Bishwas, "Comparative analysis of diverse methodologies for portfolio optimization leveraging quantum annealing techniques," *arXiv preprint*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.02599>
- [4] K. Erwin and A. Engelbrecht, "Meta-heuristics for portfolio optimization," Springer Nature, Accessed: May 03, 2025. [Online]. Available: https://www.researchgate.net/publication/370181381_Meta-heuristics_for_portfolio_optimization
- [5] "Bayesian optimization for CVaR-based portfolio optimization," *arXiv preprint*, 2025. [Online]. Available: <https://arxiv.org/html/2503.17737v1>
- [6] Q. Guo, "Review of Research on Markowitz Model in Portfolios," *www.atlantispress.com*, Apr. 29, 2022. <https://www.atlantispress.com/proceedings/icssed-22/125973810>
- [7] H. Zhang, "Effectiveness and Limitation of Markowitz Mean-variance Model: Evidence from Hang Seng Index," *Highlights in Business, Economics and Management*, vol. 8, pp. 223–230, Apr. 2023, doi: <https://doi.org/10.54097/hbem.v8i.7208>.
- [8] Dogan, S., Bezgin, M. S., & Karacayir, E. (2024b). The portfolio optimization with simulated annealing algorithm: An application of Borsa Istanbul. *Gazi İktisat Ve İşletme Dergisi*, 10(1), <https://doi.org/10.30855/gieb.2024.10.1.001>
- [9] Sinha, P., Chandwani, A., & Sinha, T. (2014). Algorithm of construction of optimum portfolio of stocks using genetic algorithm. *International Journal of Systems Assurance Engineering and Management*, 6(4), 447–465. <https://doi.org/10.1007/s13198-014-0293-7>
- [10] Das, Sourav and Chakraborty, Debjani and Mitra, Pabitra. (2024). An Adaptive Portfolio Allocation Through Bayesian Optimization. Available at SSRN: <https://ssrn.com/abstract=4916439>
- [11] J. N. Kapiamba, Berthold Ulungu, and P. K. Mubenga, "Simulated Annealing vs Genetic Algorithm to Portfolio Selection," May 01, 2015. https://www.researchgate.net/publication/384246862_Simulated_Annealing_vs_Genetic_Algorithm_to_Portfolio_Selection
- [12] W. F. Sharpe, "Mutual fund performance," *The Journal of Business*, vol. 39, pp. 119–138, 1966. [Online]. Available: <https://www.scirp.org/reference/referencespapers?referenceid=1451307>
- [13] K. V. Park, "Optimizing Portfolio Performance through Clustering and Sharpe Ratio-Based Optimization: A Comparative Backtesting Approach," *arXiv.org*, 2025, <https://arxiv.org/abs/2501.12074> (accessed May 04, 2025).
- [14] Y. Jin, R. Qu, and J. Atkin, "Constrained Portfolio Optimisation: The State-of-the-Art Markowitz Models," *5th International Conference on Operations Research and Enterprise Systems*, pp. 388–395, Feb. 2016, doi: <https://doi.org/10.5220/0005758303880395>.

- [15] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983, doi: [10.1126/science.220.4598.671](https://doi.org/10.1126/science.220.4598.671).
- [16] Y. Xiao, "Optimization of Security Investment Portfolio Based on Improved Simulated Annealing Algorithm," *International Journal of Business and Management*, vol. 6, no. 4, pp. 75–77, 2011. [Online]. Available: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=20ac719e05f68ee2bca7f487b9ffc93273187d6d>.
- [17] T.-J. Chang, N. Meade, J. E. Beasley, and Y. M. Sharaiha, "Heuristics for cardinality constrained portfolio optimisation," *Computers & Operations Research*, vol. 27, no. 13, pp. 1271–1302, 2000, doi: [10.1016/S0305-0548\(99\)00074-X](https://doi.org/10.1016/S0305-0548(99)00074-X).
- [18] P. J. M. van Laarhoven and E. H. L. Aarts, "Simulated annealing," *Simulated Annealing: Theory and Applications*, vol. 37, pp. 7–15, 1987, doi: [10.1007/978-94-015-7744-1_2](https://doi.org/10.1007/978-94-015-7744-1_2).
- [19] K. Amine, "Multiobjective Simulated Annealing: Principles and Algorithm Variants," *Advances in Operations Research*, 2019, pp. 1–13, doi: [10.1155/2019/8134674](https://doi.org/10.1155/2019/8134674).
- [20] M. E. Lundy and A. I. Mees, "Convergence of an annealing algorithm," *Mathematical Programming*, vol. 34, no. 1, pp. 111–124, 1986, doi: [10.1007/bf01582166](https://doi.org/10.1007/bf01582166).
- [21] G. Velleda Gonzales, E. Domingues dos Santos, L. Ramos Emmendorfer, L. André Isoldi, L. A. Oliveira Rocha, and E. Da Silva Diaz Estrada, "A Comparative Study of Simulated Annealing with Different Cooling Schedules for Geometric Optimization of a Heat Transfer Problem According to Constructal Design," *Scientia Plena*, vol. 11, no. 8, 2015, doi: [10.14808/sci.plena.2015.081321](https://doi.org/10.14808/sci.plena.2015.081321).
- [22] J. H. Holland, *Adaptation in Natural and Artificial Systems*, The MIT Press, 1992, doi: [10.7551/mitpress/1090.001.0001](https://doi.org/10.7551/mitpress/1090.001.0001).
- [23] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, 1989.
- [24] K. Deb and R. B. Agrawal, "Simulated Binary Crossover for Continuous Search Space," *Complex Systems*, vol. 9, no. 2, pp. 115–148, 1995.
- [25] L. J. Eshelman and J. D. Schaffer, "Real-coded Genetic Algorithms and Interval-Schemata," *Foundations of Genetic Algorithms 2*, Morgan Kaufmann, 1993.
- [26] Z. Michalewicz, *Genetic Algorithms + Data Structures = Evolution Programs*, Springer, 1996.
- [27] A. E. Eiben and J. E. Smith, *Introduction to Evolutionary Computing*, Springer, 2003.
- [28] C. A. Coello Coello, "Constraint-Handling Techniques with Evolutionary Algorithms," *Computer Methods in Applied Mechanics and Engineering*, vol. 191, no. 11–12, pp. 1245–1287, 2002.
- [29] S. Salgia, S. Vakli, and Q. Zhao, "Random Exploration in Bayesian Optimization: Order-Optimal Regret and Computational Efficiency," *ArXiv.org*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.15351>.
- [30] T. P. Minka, "Estimating a Dirichlet Distribution," Technical Report, MIT, 2000.
- [31] "The Dirichlet distribution: What is it and why is it useful?," *builtin.com*. [Online]. Available: <https://builtin.com/data-science/dirichlet-distribution>
- [32] L. V. Jospin, H. Laga, F. Boussaid, W. Buntine, and M. Bennamoun, "Hands-On Bayesian Neural Networks—A Tutorial for Deep Learning Users," *IEEE Computational Intelligence Magazine*, vol. 17, no. 2, pp. 29–48, 2022, doi: [10.1109/mci.2022.3155327](https://doi.org/10.1109/mci.2022.3155327).